

QueryTriage

A decision-driven migration engine
for legacy SQL

— *in active development* —

A folder of .sql files in — a dependency DAG, a quantified
risk report and a human-review workbook out.
Every trap found, flagged and priced.

*Validated on Microsoft's real WideWorldImporters
database project — 336 files, 218 review findings in ~22 seconds*

Malin Jacobsson

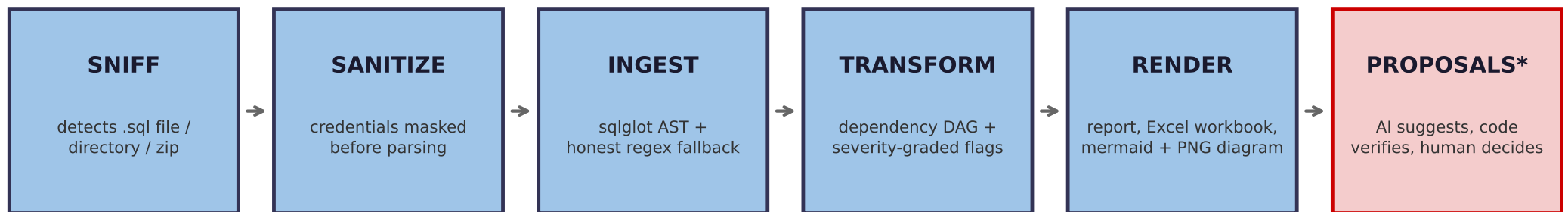
The problem

Every legacy migration starts in the same place: a database full of procedures, views and functions nobody has mapped. Circular references, dynamic SQL, deprecated syntax and dead code hide in the corpus — and nobody knows the load order.

Before anyone can even price the migration, somebody spends days just answering: what is actually IN this database?

QueryTriage answers that question in seconds — from the .sql files alone. No database connection required.

How it works



The first five stages are deterministic — same input, same output.
What the parser cannot prove is marked confidence=inferred, never silently dropped. Proposals are opt-in behind a backend interface.

Outputs: `*_canonical.json` · `*_out.json` · `*_dag.md` · `*_dag.png` ·
`*_review.md` · `*_hitl.xlsx` · `*_decisions.json` · `*_run_report.json`

vector — zoom in on this page to read the nodes

The review loop

Reviewing IS migrating — each decision mutates the graph and regenerates the artifacts, not just the report.

Every warn flag and dropped item becomes a numbered decision

decisions.json — stable ids,
survives re-runs

Click a flag — see where the problem lives

canonical source slice +
the DAG node it lands on

Buttons say the outcome, not the process

Break cycle · Dead code — drop object ·
Rewrite planned · Confirmed external

Zero-dependency, offline, runs where the data lives

the Excel workbook stays as
the exportable record

Drop a circular edge or a dead object — the DAG rebuilds in ~1 second and the cycle visibly disappears.

The DAG — before / after a decision

Layered dependency view

topological order — what can load first is on top

read / write / call edges

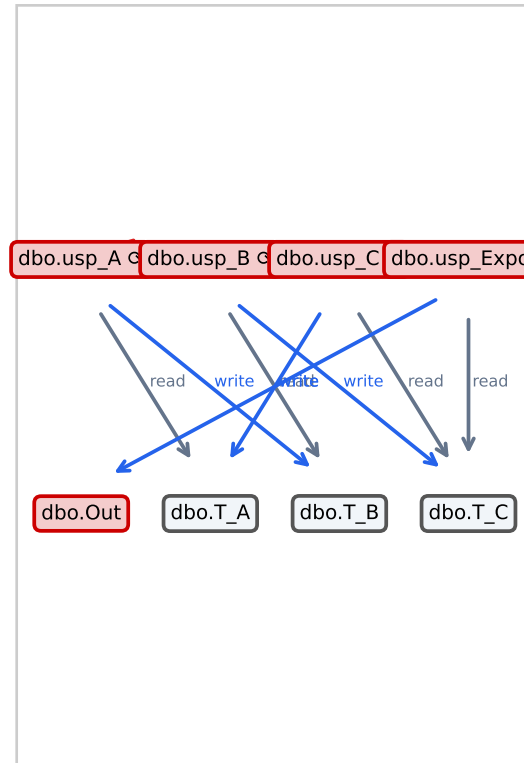
solid when parsed from the AST,
dashed when inferred

Flagged objects tinted red

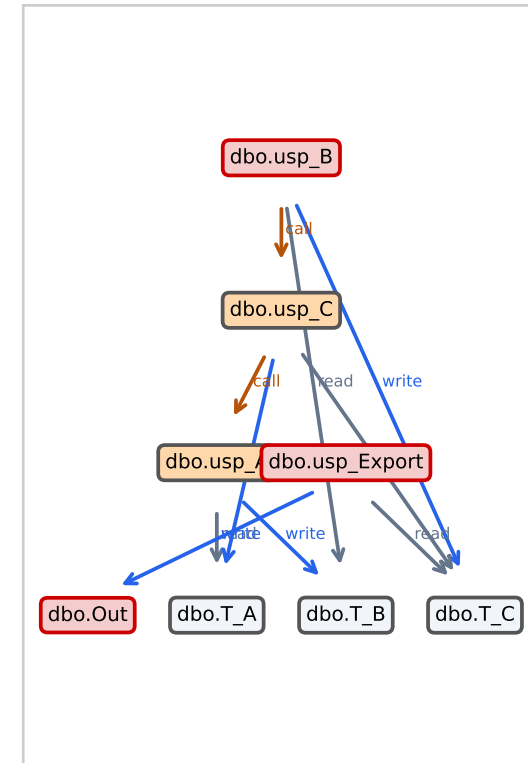
cycles marked ∞ — the map doubles
as a risk radar

Rebuilt after every decision

ignore an edge and the cycle
visibly breaks — watch it untangle



BEFORE — 1 cycle



AFTER — cycle broken

decision: ignore the edge `usp_A -> usp_B` —
cycles recomputed to 0, the flag resolves itself

What's left for a human

The honest backlog — what static analysis provably cannot resolve, counted live.

Dynamic SQL

EXEC of a variable builds its target at runtime — flagged, not guessed. A human reads the string.

External & missing references

objects outside the corpus (linked servers, cross-db, framework-owned) — confirmed, or cut as dead links.

Deprecated syntax & SELECT *

rewrite decisions — small but mandatory before any platform move.

Dead code

unreferenced objects and write-never-read tables — keep for a reason, or drop deliberately.

Empty queue + empty manual work = a finished, reproducible migration spec.

Design principles

Nothing is hidden

Objects the parser sees but cannot fully parse are kept via regex fallback and marked inferred — the chaos stays visible.

Provenance is structural

Observed facts, inferred edges, dropped constructs and AI proposals are separate channels — what the engine KNOWS vs GUESSES.

AI proposes — never decides

An optional backend may suggest resolutions. Code verifiers check every proposal against the model; the human approves.

Runs where the data lives

Deterministic, offline-capable, no telemetry — reads files, never connects to the database.